



remlax: Automatic Differentiation for Restricted Maximum Likelihood, with a Structure Catalogue and a Choice of Engine

Nicolas Salas
INRAE

Abstract

Fitting linear mixed models by restricted maximum likelihood is routine in plant and animal breeding, but the software that covers the structures breeders need—separable spatial fields, unstructured multi-trait covariances, reduced-rank factor analytic models—is either proprietary or requires the user to derive and code a likelihood by hand. **remlax** is a free implementation that forms the marginal covariance explicitly and differentiates it automatically, so that any structure added to its catalogue receives its gradient for free. The same model specification is fitted by either of two engines: a dense engine that differentiates V and runs unchanged on CPU or GPU, and a sparse engine built on the mixed model equations for the subset of structures whose inverse is sparse and known in closed form. Which engine is faster is not a matter of problem size: we show that it is decided by where the density sits, and we give a structural criterion that can be evaluated before any numerical work.

Keywords: REML, mixed models, automatic differentiation, JAX, variance components, spatial models, R, Python.

1. Introduction

2. Model and parameterisation

We consider the linear mixed model

$$y = X\beta + \sum_{k=1}^K Z_k u_k + e, \quad u_k \sim \mathcal{N}(0, G_k), \quad e \sim \mathcal{N}(0, R), \quad (1)$$

Form	Meaning	Parameters
iid	scalar variance	1
diag	heterogeneous variances, no correlation	t
us	unstructured	$t(t+1)/2$
fa	factor analytic, m factors	$tm + t$
rr	reduced rank, m factors	tm
chol	Cholesky parameterisation	$t(t+1)/2$
ante	antedependence	$2t - 1$
corh	common correlation, heterogeneous variances	$t + 1$
fixed	supplied, not estimated	0

Table 1: Covariance forms available for Σ .

with marginal covariance $V = \sum_k Z_k G_k Z_k' + R$. The restricted log-likelihood, up to an additive constant,

$$-2 \log L_R(\theta) = \log |V| + \log |X'V^{-1}X| + y'Py, \quad P = V^{-1} - V^{-1}X(X'V^{-1}X)^{-1}X'V^{-1}, \quad (2)$$

is a function of the variance parameters θ alone.

2.1. One hand-written derivative rule

The design decision from which everything else follows is that **remlax** does not differentiate through the Cholesky factorisation of V . It supplies the derivative of (2) with respect to V in closed form,

$$\frac{\partial (-2 \log L_R)}{\partial V} = P - (Py)(Py)', \quad (3)$$

and lets automatic differentiation propagate that single rule backwards through whatever code built V from θ .

Two consequences matter. Memory: reverse-mode differentiation through a Cholesky factorisation stores the intermediates of the factorisation, whereas (3) needs P and Py only, so three $n \times n$ matrices are live at once rather than a dozen. Genericity: a new covariance structure is a new way of building V , and it receives its gradient without any further derivation. The catalogue of Section 3 is a direct consequence—adding a structure is writing its forward construction.

3. The structure catalogue

A term contributes $Z_k(\Sigma_k \otimes C_k)Z_k'$ to V , where Σ_k is a covariance between traits and C_k a correlation between levels. **remlax** parameterises the two independently: nine forms for Σ and twenty-five for C , so their product covers the models used in multi-trait and spatial analyses without a combinatorial special case for each pairing.

The level structures C cover autoregressive and moving-average families (**ar1**, **ar2**, **ar3**, **sar**, **ma1**, **ma2**, **arma**), isotropic and anisotropic metric kernels (**exp**, **gau**, **iexp**, **igau**, **ieuc**, **sph**, **cir**, **aexp**, **agau**, **mtrn**), correlation forms (**cor**, **corb**, **corg**), the separable product **ar1ar1**, and the degenerate cases **id**, **fixed** and **own**.

3.1. Reported scales

Every family is estimated on an unconstrained scale and reported on its natural one. This matters for interpretation: a correlation is reported as the rescaled value actually present in C , not as the transformed parameter the optimiser moves. For the intrinsic metric kernels the sign of the range parameter is not identified— θ and $-\theta$ give the same C —and the reported value is taken positive.

4. Two engines for one specification

4.1. The dense engine

The dense engine forms V explicitly, factorises it, and applies (3). It is written in JAX in double precision and runs unchanged on CPU or GPU. Its cost is $O(n^3)$ per evaluation and it is indifferent to how sparse the model is.

4.2. The sparse engine

The sparse engine never forms V . It works on the mixed model equations,

$$C = \begin{pmatrix} X'R^{-1}X & X'R^{-1}Z \\ Z'R^{-1}X & Z'R^{-1}Z + G^{-1} \end{pmatrix}, \quad (4)$$

which contain G^{-1} and R^{-1} rather than G and R . It is built on **RTMB** (Kristensen 2025), declaring both β and u as random so that the Laplace approximation—exact for a linear Gaussian model—supplies the sparse factorisation, the log-determinant and the exact gradient.

Its scope is a declared, testable rule rather than a heuristic: a model is in scope when every level structure has a sparse inverse known in closed form. Three qualify (`id`, `ar1`, `ar1ar1`), together with a precision supplied directly by the user, and three covariance forms (`iid`, `diag`, `us`). Everything else stays on the dense engine. A consequence of the rule is that the sparse engine never inverts anything: the retained structures have analytic inverses and analytic log-determinants.

4.3. Declaring a term by its precision

A term may be declared by K , its between-level covariance, or by K^{-1} , its precision. The second is the sparse route for a pedigree: Henderson's rules give A^{-1} directly, with about five non-zeros per individual, without ever forming A or factorising it. Supplying K does not permit this, since its Cholesky factor is dense and the inverse of a sparse matrix is generally full. Supplying both is refused, and a precision that is itself heavily filled—the genomic case—raises a warning, since sparsity buys nothing there.

5. Validation

Validation is organised around what a failure would have meant, not around test counts. Three levels are used, and the third is the one that found defects the other two could not.

5.1. Internal agreement

tests/R/test_remlax_tmb_parity.R checks that the two engines read θ identically: that $\Sigma(\theta)$ agrees for **iid**, **diag** and **us**; that the closed-form **ar1** and **ar1ar1** precisions are exact inverses of the correlation matrices the dense engine builds, verified by forming $K^{-1}K$ rather than by inspecting a sparsity pattern; and that the analytic log-determinants agree with numerical ones. Thirty checks, largest discrepancy 3.6×10^{-15} .

5.2. Cross-engine agreement, in both directions

tests/R/test_remlax_tmb_vs_dense.R evaluates each engine at the other’s optimum. Both directions are needed: an engine could have a correct surface along the other’s path and be wrong elsewhere. Sixteen checks over four designs agree to 10^{-12} on cross-evaluation and 6×10^{-9} on the attained optima.

5.3. An arbiter beholden to neither engine

Two engines that agree prove less than one engine checked against an independent computation. *tests/R/test_remlax_tmb_prec.R* carries a REML likelihood written out in dense linear algebra, and it settled three disagreements during development that engine-to-engine comparison could not. In one of them it identified the sparse engine as wrong; in another, the dense engine; in the third, the test design itself.

The defect it caught in the precision path deserves reporting, because it is the class of defect an estimate-based test cannot see. The objective read a user-supplied log-determinant field and silently fell back to zero when absent, so $\log |K|$ was simply missing. Being constant, the omission left the gradient untouched: the parameter estimates agreed to five digits and only the likelihood was wrong—hence every model comparison, likelihood ratio test and information criterion computed from it.

5.4. Agreement with established software

Against **asreml** 4.2 (Butler, Cullis, Gilmour, Gogel, and Thompson 2023), **lme4** (Bates, Mächler, Bolker, and Walker 2015) and **sommer** (Covarrubias-Pazarán 2016), in a container where all are installed: restricted log-likelihoods agree to 10^{-5} in **asreml**’s own convention, variance components to the same order, and heritabilities with their delta-method standard errors.

BLUPs are checked separately, in *tests/R/test_remlax_asreml_blup.R*, because they are the output a breeder actually uses and no earlier suite touched them. Matching levels by name rather than by position, all 60 predictions agree with **asreml** to 3.2×10^{-6} , with a correlation of 1 to twelve digits, and the fixed effect to ten.

That test also records a gap rather than a disagreement: **remlax** computes BLUPs but does not expose their prediction error variances, where **asreml** reports a standard error and a z ratio for every effect. Without them a candidate ranking carries no measure of reliability, so this is a required addition, and the information is available since the solver already factorises the coefficient matrix.

5.5. A conditioning limit, made deliberate

The mixed model equations (4) contain R^{-1} , so the sparse engine degrades as the residual

variance approaches zero. Rather than hope an optimiser visits that region, the check imposes θ and reports the degradation, which is monotone: at $\sigma_e^2 = 3.4 \times 10^{-4}$ the two engines agree to 3.8×10^{-11} , and at 3.8×10^{-11} they agree only to 1.7×10^{-4} . This is structural to the formulation, and it is why **asrem1** parameterises the ratio of variances instead.

6. Performance

6.1. Measurement protocol

Three points of protocol are stated here because each of them changed a conclusion during development.

Every GPU figure reported below is measured on a *whole* A100. Earlier sweeps used a one-seventh MIG slice, and we had assumed the slice figures could be treated as a fixed pessimistic bound. They cannot: measured on the same script, the slice-to-card ratio is not constant.

n	Slice (s)	Whole card (s)	Ratio
250	0.32	0.28	1.16
1000	0.40	0.32	1.23
2000	1.26	0.65	1.94
4000	6.39	1.73	3.69
8000	94.19	15.49	6.08
12000	294.75	41.88	7.04

Table 2: A one-seventh slice against the whole card, same script. The ratio rises from 1.16 to 7.04 and never approaches the compute-unit ratio of 15.4: below $n \approx 2000$ the fixed costs dominate and a bigger card buys almost nothing, while above $n = 8000$ the factorisation is bandwidth-bound rather than unit-bound.

One practical consequence is worth stating for users of shared hardware: for several independent models of moderate size, seven slices beat one whole card, and the reverse holds beyond $n = 8000$.

Second, each grid cell is measured in a *fresh process*. A sweep that measures all cells in one process reuses compiled programs across cells, so execution order influences the result—in an earlier sweep the compilation column fell from 79 s to 2.7 s on the late cells for that reason.

Third, cell names below are written out rather than abbreviated: a *case* is the structure of the data (a separable **ar1** field, or a full genomic kinship), and an *engine* is what fits it. The two are crossed, so a structurally dense case can be forced onto the sparse engine—a cell its declared scope excludes, measured here to price the wrong choice rather than assert it.

6.2. Fixed costs, and an unfavourable comparison

A reader comparing **rem1ax** to established software on a small model will find it slower, and by a wide margin. We report that measurement first, because the decomposition behind it is the informative part.

On a single-trait model with 720 observations, 60 genotypes and two variance parameters,

asrem1 fits in 0.080 s. **rem1ax** takes 3.40 s called from R. Decomposing that figure:

Quantity measured	Time	Ratio to asrem1
Wall time, called from R	3.398 s	42.5
Fit in process	1.329 s	16.6
Internal time reported by the solver	0.598 s	7.5
Algebraic cost alone (9×0.0137 s)	0.123 s	1.5

Table 3: Where the time goes on a small model. The linear algebra is within a factor of 1.5; everything above it is fixed cost.

Compilation, the finite-difference Hessian and the BLUPs account for 90.7 % of the in-process fit, and the R interface adds 2.8 s of out-of-process invocation because it launches a Python process per fit. On a model with two parameters and nine iterations there is nothing to amortise these costs against.

This unifies the section. Fixed costs are amortised over evaluations, and the number of evaluations is governed by the number of parameters, so **rem1ax** becomes competitive exactly when a model is large enough in parameters to pay for its own setup—the same criterion as for compilation in Section 6. It also identifies the most valuable optimisation in the package, which is not algebraic: a persistent worker process reused across fits would remove the 2.8 s entirely, and it matters most for the serial-fitting workloads (likelihood ratio tests over many reduced models) where hundreds of fits are performed.

6.3. Compilation is a purchasable cost, not a fixed one

rem1ax compiles the model to a program before evaluating it, and that compilation runs on the CPU whichever device the program targets. On a dense-kinship model with $n = 2000$ and $q = 1000$:

Cores	CPU backend	GPU backend	Backend gap
2	75.99 s	79.89 s	5.1 %
8	20.95 s	21.49 s	2.6 %
16	10.88 s	12.24 s	12.6 %

Table 4: Compilation time against core count. Eight times the cores divide the cost by 7.0—87 % parallel efficiency—while the target device changes it by at most 13 %.

Two practical consequences follow, and the second is counter-intuitive. Compilation is amortised over evaluations, so it is negligible for a model with many parameters and dominant for a model with few: the break-even is counted in evaluations, not in problem size. And a GPU job should request many CPU cores; fitting several models concurrently on separate GPU slices with few cores each is slower than fitting them sequentially with all cores.

6.4. Where each engine wins

For a structurally sparse model the sparse engine wins by a wide margin: at $n = 4000$ it fits in 2.6 s against 608 s for the dense engine on CPU, a factor of 230. For dense genomic kinship

the picture is not the one the folklore suggests—the sparse engine still wins most cells, and loses only when the number of effects is large relative to the number of observations. What separates the two is that the sparse engine’s cost is governed by q and is flat in n (2.43, 2.43 and 2.39 s per evaluation at $q = 500$ for $n = 1000, 2000, 4000$), while the dense engine’s cost grows with n .

6.5. The empty cell

The sparse engine has no GPU path, and this is not an omission. Sparse factorisation goes through **CHOLMOD**, which is CPU-only, and the reason is structural: irregular memory access and the sequential dependencies of the elimination tree give it low arithmetic intensity, where a dense product reaches close to peak. Sparse and GPU are therefore mutually exclusive rather than two independent axes, which turns the choice of engine into an arbitrage.

7. Choosing an engine

The criterion is structural and costs no numerical work. What decides is not the size of the problem but where the density sits. In (4), a dense G fills its own block, from which fill propagates through elimination; a dense R fills $Z'R^{-1}Z$ for every term at once, including terms whose precision is diagonal. The prediction, which the accompanying screening tests at matched parameter counts, is that an unstructured residual disqualifies the sparse route where a dense genomic kinship does not.

8. Outlook

Three directions are identified by measurement rather than by preference.

Relationship matrices currently enter the compiled program as constants—the compiler reports 2.17 GB captured on a real multivariate analysis—so the program is recompiled for every size. Passing them as arguments would compile once and reuse, and the gain is measurable against the figures of Section 6.

Prediction error variances for BLUPs are computed but not yet confronted with a reference implementation.

Finally, the sparse route is limited by the CPU-only factorisation. A matrix-free formulation—for Markov random field representations of spatial fields in particular—would restore a GPU path, at the cost of a stochastic gradient and hence of a different stopping rule.

References

- Bates D, Mächler M, Bolker B, Walker S (2015). “Fitting Linear Mixed-Effects Models Using lme4.” *Journal of Statistical Software*, **67**(1), 1–48.
- Butler D, Cullis B, Gilmour A, Gogel B, Thompson R (2023). *ASReml-R Reference Manual Version 4.2*. VSN International.

8rem_{lax}: *Automatic Differentiation for Restricted Maximum Likelihood, with a Structure Catalogue and a Ch*

Covarrubias-Pazarán G (2016). “Genome-Assisted Prediction of Quantitative Traits Using the R Package sommer.” *PLOS ONE*, **11**(6).

Kristensen K (2025). *RTMB: R Bindings for TMB*. R package version 1.9.

Affiliation:

Nicolas Salas

E-mail: nicolas.salas@supagro.fr